Tutorial completo de funciones y procedimientos almacenados en PL/pgSQL (PostgreSQL 18)

Este tutorial parte de cero y avanza hasta un nivel profesional. Usaremos el esquema `tienda` que proporcionaste como base de datos de práctica. El objetivo es que domines todas las variantes de entrada y salida de parámetros, cursores, JSON, agregaciones, ventanas y subconsultas dentro de funciones PL/pgSQL.

Convención de nombres obligatoria en este tutorial: todas las variables declaradas (sean `OUT`, `RETURNS TABLE`, `DECLARE`, etc.) llevarán el prefijo `v_` seguido del nombre descriptivo. Ejemplos: `v_linea_id`, `v_pedido_id`, `v_total_importe`, `v_producto_nombre`.

---

1. Fundamentos: funciones vs. procedimientos

1.1 ¿Qué es PL/pgSQL?

PL/pgSQL es el lenguaje procedural por defecto de PostgreSQL. Permite agrupar bloques de cómputo y consultas SQL dentro del servidor, eliminando la ida y vuelta entre cliente y servidor. Las funciones creadas con PL/pgSQL pueden usarse en cualquier lugar donde se use una función incorporada: en `SELECT`, en índices, en `WHERE`, etc.

1.2 Función

Una función **siempre devuelve un valor** (o un conjunto). Se invoca con `SELECT funcion(...)`.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_total_pedido(v_pedido_id INTEGER)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC(10,2);
BEGIN
    SELECT COALESCE(SUM(cantidad * precio_unit), 0)
    INTO v_total
    FROM tienda.lineas_pedido
    WHERE pedido_id = v_pedido_id;

    RETURN v_total;
END;
$$;
```

Enunciado: Calcular el importe total de un pedido sumando sus líneas.

Resultado:

sql

```
SELECT tienda.fn_total_pedido(1);
-- 158.90
```

1.3 Procedimiento

Un procedimiento **no devuelve valor** (o usa parámetros `OUT`/`INOUT`). Se invoca con `CALL`.

sql

```
CREATE OR REPLACE PROCEDURE tienda.pr_marcar_pedido_pagado(v_pedido_id INTEGER)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE tienda.pedidos
    SET estado = 'pagado'
    WHERE pedido_id = v_pedido_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Pedido % no encontrado', v_pedido_id;
    END IF;
END;
$$;
```

Enunciado: Cambiar el estado de un pedido a 'pagado'.

Resultado:

sql

```
CALL tienda.pr_marcar_pedido_pagado(1);
-- NOTICE:  Pedido 1 actualizado (implícito en el UPDATE)
```

---

2. Variantes de parámetros de entrada y salida

PL/pgSQL ofrece múltiples formas de recibir y devolver datos. Todas las variables de salida seguirán la convención `v_`.

2.1 Parámetros posicionales ($1, $2, …)

Cuando no se nombran los parámetros, PostgreSQL los referencia internamente como `$1`, `$2`, etc.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_sumar(a INTEGER, b INTEGER)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN $1 + $2;
END;
$$;
```

Enunciado: Sumar dos números usando referencias posicionales.

Resultado:

sql

```
SELECT tienda.fn_sumar(5, 7);
```

```
-- 12
```

2.2 Parámetros con nombre (IN)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_buscar_producto(v_nombre TEXT)
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT producto_id, nombre, precio
    FROM tienda.productos
    WHERE nombre ILIKE '%' || v_nombre || '%'
      AND activo = TRUE;
END;
$$;
```

Enunciado: Buscar productos por coincidencia parcial de nombre.

Resultado:

sql

```
SELECT * FROM tienda.fn_buscar_producto('Ratón');
-- v_producto_id | v_nombre          | v_precio
-- 2             | Ratón inalámbrico | 39.50
```

2.3 Parámetros OUT

Los parámetros `OUT` permiten devolver múltiples valores sin declarar un tipo compuesto explícito.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_info_pedido(
    v_pedido_id INTEGER,
    OUT v_cliente_nombre TEXT,
    OUT v_total NUMERIC,
    OUT v_estado TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT c.nombre, COALESCE(SUM(lp.cantidad * lp.precio_unit), 0), p.estado
    INTO v_cliente_nombre, v_total, v_estado
    FROM tienda.pedidos p
    JOIN tienda.clientes c ON c.cliente_id = p.cliente_id
    LEFT JOIN tienda.lineas_pedido lp ON lp.pedido_id = p.pedido_id
    WHERE p.pedido_id = v_pedido_id
    GROUP BY c.nombre, p.estado;
END;
$$;
```

Enunciado: Obtener nombre del cliente, total del pedido y estado en una sola llamada.

Resultado:

sql

```
SELECT * FROM tienda.fn_info_pedido(1);
```

```
-- v_cliente_nombre | v_total | v_estado
-- Juan Pérez       | 158.90  | pendiente
```

2.4 Parámetros INOUT

Similar a `OUT`, pero el valor inicial se recibe y se puede modificar.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_aplicar_descuento(
    v_total INOUT NUMERIC,
    v_porcentaje NUMERIC DEFAULT 10
)
LANGUAGE plpgsql
AS $$
BEGIN
    v_total := v_total * (1 - v_porcentaje / 100.0);
END;
$$;
```

Enunciado: Aplicar un descuento porcentual a un importe.

Resultado:

sql

```
SELECT tienda.fn_aplicar_descuento(200.00, 15);
-- 170.00
```

2.5 RETURNS TABLE vs. SETOF RECORD vs. %ROWTYPE vs. RECORD

Tipo de retorno	Uso	Ventaja
`RETURNS TABLE`	Columnas definidas explícitamente	Claridad y tipado fuerte
`RETURNS SETOF record`	Columnas definidas por el llamador	Flexibilidad total
`%ROWTYPE`	Estructura de una tabla concreta	Sincronización automática con la tabla
`RECORD`	Estructura dinámica	Máxima flexibilidad, sin verificación en compilación

RETURNS TABLE

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_caros(v_umbral NUMERIC DEFAULT
100)
RETURNS TABLE (
    v_producto_id INTEGER,
    v_nombre TEXT,
    v_precio NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT producto_id, nombre, precio
    FROM tienda.productos
    WHERE precio > v_umbral
    ORDER BY precio DESC;
```

```
END;
$$;
```

Resultado:

sql

```
SELECT * FROM tienda.fn_productos_caros(100);
-- v_producto_id | v_nombre     | v_precio
-- 3             | Monitor 27"  | 249.00
```

SETOF RECORD

El llamador debe especificar la estructura al invocar.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_resumen_pedidos()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT p.pedido_id, c.nombre, p.estado, p.fecha_pedido::DATE
    FROM tienda.pedidos p
    JOIN tienda.clientes c ON c.cliente_id = p.cliente_id;
END;
$$;
```

Resultado:

sql

```
SELECT * FROM tienda.fn_resumen_pedidos()
AS t(v_pedido_id INTEGER, v_cliente TEXT, v_estado TEXT, v_fecha DATE);
```

%ROWTYPE

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_obtener_producto(v_producto_id INTEGER)
RETURNS tienda.productos
LANGUAGE plpgsql
AS $$
DECLARE
    v_producto tienda.productos%ROWTYPE;
BEGIN
    SELECT * INTO v_producto
    FROM tienda.productos
    WHERE producto_id = v_producto_id;

    RETURN v_producto;
END;
$$;
```

Resultado:

sql

```
SELECT * FROM tienda.fn_obtener_producto(1);
-- producto_id | nombre           | precio | stock | activo
-- 1           | Teclado mecánico | 79.90  | 50    | t
```

---

3. Cursores explícitos e implícitos

3.1 Cursor implícito (FOR ... IN)

El FOR recorre implícitamente el resultado de una consulta. No requiere DECLARE, OPEN, FETCH ni CLOSE.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_listar_clientes()
RETURNS TABLE (v_cliente_id INTEGER, v_nombre TEXT, v_email TEXT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN
        SELECT cliente_id, nombre, email
        FROM tienda.clientes
        ORDER BY cliente_id
    LOOP
        v_cliente_id := v_registro.cliente_id;
        v_nombre := v_registro.nombre;
        v_email := v_registro.email;
        RETURN NEXT;
    END LOOP;
END;
$$;
```

Enunciado: Listar todos los clientes usando un cursor implícito y RETURN NEXT.

Resultado:

sql

```
SELECT * FROM tienda.fn_listar_clientes();
-- v_cliente_id | v_nombre    | v_email
-- 1            | Juan Pérez  | juan@example.com
-- 2            | María López | maria@example.com
```

3.2 Cursor explícito con variables OUT

Un cursor explícito se declara con CURSOR FOR, se abre con OPEN, se recorre con FETCH y se cierra con CLOSE.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_por_estado(
    v_estado TEXT,
    OUT v_pedido_id INTEGER,
    OUT v_cliente TEXT,
    OUT v_fecha TIMESTAMP
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_cursor CURSOR FOR
        SELECT p.pedido_id, c.nombre, p.fecha_pedido
        FROM tienda.pedidos p
        JOIN tienda.clientes c ON c.cliente_id = p.cliente_id
```

```
        WHERE p.estado = v_estado;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_pedido_id, v_cliente, v_fecha;
        EXIT WHEN NOT FOUND;
        RETURN NEXT;
    END LOOP;
    CLOSE v_cursor;
END;
$$;
```

Enunciado: Listar pedidos filtrados por estado usando cursor explícito con parámetros OUT.

Resultado:

sql

```
SELECT * FROM tienda.fn_pedidos_por_estado('pendiente')
AS t(v_pedido_id INTEGER, v_cliente TEXT, v_fecha TIMESTAMP);
```

3.3 Cursor explícito con %ROWTYPE (sin OUT)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_clientes_cursor_rowtype()
RETURNS TABLE (v_cliente_id INTEGER, v_nombre TEXT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_cliente tienda.clientes%ROWTYPE;
    v_cursor CURSOR FOR SELECT * FROM tienda.clientes;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_cliente;
        EXIT WHEN NOT FOUND;

        v_cliente_id := v_cliente.cliente_id;
        v_nombre := v_cliente.nombre;
        RETURN NEXT;
    END LOOP;
    CLOSE v_cursor;
END;
$$;
```

Enunciado: Recorrer clientes usando `%ROWTYPE` como variable de fila del cursor.

Resultado: Lista de clientes con sus IDs y nombres.

---

4. JSON y JSONB en PL/pgSQL

PostgreSQL 18 acelera el procesamiento de cadenas JSON largas mediante SIMD . Las funciones JSON son fundamentales para APIs y reportes.

4.1 json_agg + json_build_object

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedido_json(v_pedido_id INTEGER)
RETURNS JSON
```

```
LANGUAGE plpgsql
AS $$
DECLARE
    v_resultado JSON;
BEGIN
    SELECT json_build_object(
        'pedido_id', p.pedido_id,
        'cliente', c.nombre,
        'estado', p.estado,
        'lineas', json_agg(
            json_build_object(
                'producto', pr.nombre,
                'cantidad', lp.cantidad,
                'precio_unit', lp.precio_unit,
                'subtotal', lp.cantidad * lp.precio_unit
            )
        )
    )
    INTO v_resultado
    FROM tienda.pedidos p
    JOIN tienda.clientes c ON c.cliente_id = p.cliente_id
    JOIN tienda.lineas_pedido lp ON lp.pedido_id = p.pedido_id
    JOIN tienda.productos pr ON pr.producto_id = lp.producto_id
    WHERE p.pedido_id = v_pedido_id
    GROUP BY p.pedido_id, c.nombre, p.estado;

    RETURN v_resultado;
END;
$$;
```

Enunciado: Generar un JSON completo de un pedido con sus líneas anidadas.

Resultado:

json

```
{
  "pedido_id": 1,
  "cliente": "Juan Pérez",
  "estado": "pendiente",
  "lineas": [
    {"producto": "Teclado mecánico", "cantidad": 1, "precio_unit": 79.90,
"subtotal": 79.90},
    {"producto": "Ratón inalámbrico", "cantidad": 2, "precio_unit": 39.50,
"subtotal": 79.00}
  ]
}
```

4.2 to_json / to_jsonb / row_to_json

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_producto_jsonb(v_producto_id INTEGER)
RETURNS JSONB
LANGUAGE plpgsql
AS $$
DECLARE
    v_producto tienda.productos%ROWTYPE;
BEGIN
    SELECT * INTO v_producto FROM tienda.productos WHERE producto_id =
v_producto_id;
    RETURN to_jsonb(v_producto);
END;
```

```
$$;
```

Enunciado: Convertir una fila completa de producto a JSONB usando `to_jsonb`.

Resultado:

json

```
{"producto_id": 1, "nombre": "Teclado mecánico", "precio": 79.90, "stock": 50,
"activo": true}
```

4.3 query execute (SQL dinámico)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_consulta_dinamica(v_tabla TEXT)
RETURNS JSONB
LANGUAGE plpgsql
AS $$
DECLARE
    v_sql TEXT;
    v_resultado JSONB;
BEGIN
    v_sql := format('SELECT jsonb_agg(t) FROM tienda.%I t', v_tabla);
    EXECUTE v_sql INTO v_resultado;
    RETURN v_resultado;
END;
$$;
```

Enunciado: Devolver todos los registros de una tabla como un array JSONB, usando SQL dinámico con `format` y `%I` (identificador seguro).

Resultado:

sql

```
SELECT tienda.fn_consulta_dinamica('clientes');
-- [{"cliente_id":1,"nombre":"Juan Pérez",...}, ...]
```

---

5. Funciones de agregación y ventana

5.1 Agregación con FILTER

PostgreSQL permite filtrar qué filas participan en cada agregación usando `FILTER (WHERE ...)`. Esto evita subconsultas y escanea la tabla una sola vez .

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_estadisticas_pedidos()
RETURNS TABLE (
    v_total_pedidos BIGINT,
    v_pedidos_pendientes BIGINT,
    v_importe_total NUMERIC,
    v_importe_pendiente NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        COUNT(*),
```

```
        COUNT(*) FILTER (WHERE p.estado = 'pendiente'),
        COALESCE(SUM(lp.cantidad * lp.precio_unit), 0),
        COALESCE(SUM(lp.cantidad * lp.precio_unit) FILTER (WHERE p.estado =
'pendiente'), 0)
    FROM tienda.pedidos p
    LEFT JOIN tienda.lineas_pedido lp ON lp.pedido_id = p.pedido_id;
END;
$$;
```

Enunciado: Calcular estadísticas de pedidos en una sola pasada: total, pendientes, importe total e importe pendiente.

Resultado:

sql

```
SELECT * FROM tienda.fn_estadisticas_pedidos();
-- v_total_pedidos | v_pedidos_pendientes | v_importe_total |
v_importe_pendiente
-- 2               | 2                    | 407.90          | 407.90
```

5.2 Funciones de ventana

Las funciones de ventana permiten calcular rankings, acumulados y comparaciones sin colapsar filas.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_ranking_productos()
RETURNS TABLE (
    v_producto TEXT,
    v_precio NUMERIC,
    v_ranking BIGINT,
    v_acumulado NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        nombre,
        precio,
        RANK() OVER (ORDER BY precio DESC),
        SUM(precio) OVER (ORDER BY precio DESC ROWS UNBOUNDED PRECEDING)
    FROM tienda.productos
    WHERE activo = TRUE;
END;
$$;
```

Enunciado: Rankear productos por precio y calcular el acumulado descendente.

Resultado:

sql

```
SELECT * FROM tienda.fn_ranking_productos();
-- v_producto       | v_precio | v_ranking | v_acumulado
-- Monitor 27"      | 249.00   | 1         | 249.00
-- Teclado mecánico | 79.90    | 2         | 328.90
-- Ratón inalámbrico| 39.50    | 3         | 368.40
```

---

6. Subconsultas y consultas anidadas

6.1 Subconsulta escalar en RETURN QUERY

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_clientes_mejor_pedido()
RETURNS TABLE (
    v_cliente TEXT,
    v_mejor_importe NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.nombre,
        (SELECT COALESCE(SUM(lp.cantidad * lp.precio_unit), 0)
         FROM tienda.lineas_pedido lp
         WHERE lp.pedido_id = p.pedido_id) AS total
    FROM tienda.clientes c
    JOIN tienda.pedidos p ON p.cliente_id = c.cliente_id
    ORDER BY total DESC;
END;
$$;
```

Enunciado: Listar clientes con el importe de su pedido más caro usando subconsulta escalar.

Resultado:

sql

```
SELECT * FROM tienda.fn_clientes_mejor_pedido();
-- v_cliente    | v_mejor_importe
-- María López  | 249.00
-- Juan Pérez   | 158.90
```

6.2 Subconsulta en FROM (tabla derivada)

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_productos_mas_vendidos()
RETURNS TABLE (v_producto TEXT, v_unidades BIGINT)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT pr.nombre, sub.total_unidades
    FROM (
        SELECT producto_id, SUM(cantidad) AS total_unidades
        FROM tienda.lineas_pedido
        GROUP BY producto_id
    ) sub
    JOIN tienda.productos pr ON pr.producto_id = sub.producto_id
    ORDER BY sub.total_unidades DESC;
END;
$$;
```

Enunciado: Obtener productos ordenados por unidades vendidas usando una subconsulta en `FROM`.

Resultado:

sql

```
SELECT * FROM tienda.fn_productos_mas_vendidos();
-- v_producto        | v_unidades
-- Ratón inalámbrico | 2
-- Teclado mecánico  | 1
-- Monitor 27"       | 1
```

---

7. Resumen de buenas prácticas

1. **Usa `v_` como prefijo** en todas las variables para distinguirlas de columnas y parámetros.
2. **`RETURNS TABLE` es preferible a `SETOF RECORD`** cuando la estructura es fija: el tipado fuerte evita errores en tiempo de ejecución.
3. **`%ROWTYPE`** es ideal cuando la función devuelve o manipula una fila completa de una tabla.
4. **`RECORD`** solo cuando la estructura no se conoce de antemano (p. ej., resultados dinámicos).
5. **Cursores implícitos** (`FOR ... IN`) son más simples y seguros; los explícitos solo cuando necesites control fino (FETCH direccional, actualizaciones posicionadas).
6. **`FILTER (WHERE ...)`** en agregaciones evita subconsultas y escanea la tabla una sola vez.
7. **JSONB** es preferible a JSON para almacenamiento y consultas; `json_agg` + `json_build_object` son la pareja ideal para reportes anidados.
8. **`EXECUTE` con `format('%I', ...)`** para identificadores dinámicos; evita inyección SQL.
9. **PostgreSQL 18** acelera JSON con SIMD y mejora el rendimiento general; el código PL/pgSQL existente se beneficia sin cambios .

   Este tutorial cubre todas las variantes solicitadas. Cada ejemplo está listo para ejecutarse sobre el esquema `tienda` que proporcionaste.

   ahora explica porque , como y cuando y pon ejemplos de return next y return next v_rec, o return v_nombre,v_email...etc

   Encontrados 20 resultados web

   `RETURN NEXT` es la instrucción central de las funciones PL/pgSQL que devuelven conjuntos de resultados (`SETOF`). No "devuelve" ni "sale" de la función de inmediato; su única función es **añadir una fila al conjunto de resultados** que se está construyendo. Después de ejecutar `RETURN NEXT`, la función continúa ejecutando la siguiente instrucción hasta encontrar un `RETURN;` sin argumentos o llegar al final de la función, momento en el que se devuelve todo el conjunto de resultados acumulado.

   A continuación se explica en detalle desde tres perspectivas: **por qué**, **cómo** y **cuándo**, junto con ejemplos.

Por qué: La diferencia esencial entre `RETURN NEXT` y `RETURN`

Para entender `RETURN NEXT`, primero hay que distinguirlo de `RETURN`:

- **`RETURN expresión;`**: **Finaliza** la función inmediatamente y devuelve un valor. Se utiliza para funciones escalares (como `RETURNS integer`) o cuando se devuelve una sola fila.
- **`RETURN NEXT;`**: **No finaliza** la función. Solo "empaqueta" los datos de la fila actual (ya sea una variable escalar o un tipo compuesto completo) y los añade a la lista de resultados. La función continúa ejecutándose.

  Por lo tanto, si tu función está declarada para devolver `SETOF` (un conjunto de resultados), debes usar `RETURN NEXT` (o `RETURN QUERY`) para acumular filas, y finalmente usar un `RETURN;` sin parámetros para indicar el final.

Cómo: Tres formas de uso comunes

Caso 1: `RETURN NEXT v_registro` (devolver una fila completa / `RECORD`)

Esta es la forma más común y eficiente. Cuando usas `FOR ... IN ... LOOP` para recorrer una consulta, la variable del bucle es esencialmente un `RECORD` (o un `%ROWTYPE`), y `RETURN NEXT` añade directamente toda la fila al conjunto de resultados.

Requisito: La estructura de la variable debe coincidir con la estructura de columnas definida por `RETURNS SETOF`.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_pendientes()
RETURNS SETOF tienda.pedidos  -- 返回 pedidos 表的完整行结构
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.pedidos%ROWTYPE;  -- 变量名遵循 v_ 约定
BEGIN
    FOR v_registro IN
        SELECT * FROM tienda.pedidos WHERE estado = 'pendiente'
    LOOP
        RETURN NEXT v_registro;  -- 将当前行追加到结果集
    END LOOP;
    RETURN;  -- 结束函数
END;
$$;
```

Ejemplo de llamada:

sql

```
SELECT * FROM tienda.fn_pedidos_pendientes();
```

Caso 2: `RETURN NEXT v_nombre, v_email` (devolver múltiples campos escalares)

Si tu función devuelve `RETURNS TABLE(v_nombre TEXT, v_email TEXT)`, puedes construir filas usando **parámetros de salida (`OUT`)** o **variables locales**, y usar `RETURN NEXT` para enviarlas.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_listar_clientes_basico()
RETURNS TABLE (v_nombre TEXT, v_email TEXT)
```

```
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
BEGIN
    FOR v_registro IN SELECT nombre, email FROM tienda.clientes LOOP
        -- 方式 A：直接赋值给 OUT 参数，然后用无参 RETURN NEXT
        v_nombre := v_registro.nombre;
        v_email := v_registro.email;
        RETURN NEXT;  -- 保存当前 OUT 变量的值作为一行
    END LOOP;
END;
$$;
```

Nota: Si la función usa `RETURNS TABLE`, los nombres de las columnas son en realidad parámetros `OUT`. En este caso, la práctica estándar es asignar valores y luego usar un `RETURN NEXT;` **sin parámetros**, que guarda los valores actuales de todas las variables `OUT`.

Caso 3: `RETURN NEXT` combinado con lógica condicional (control preciso del flujo)

Dado que `RETURN NEXT` no interrumpe la ejecución, puedes decidir dinámicamente si añadir una fila o modificar los datos antes de añadirla dentro del bucle.

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_clientes_con_filtro(v_min_pedidos INT)
RETURNS TABLE (v_nombre TEXT, v_total_pedidos BIGINT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;
    v_contador BIGINT;
BEGIN
    FOR v_registro IN SELECT cliente_id, nombre FROM tienda.clientes LOOP
        SELECT COUNT(*) INTO v_contador
        FROM tienda.pedidos WHERE cliente_id = v_registro.cliente_id;

        IF v_contador >= v_min_pedidos THEN
            v_nombre := v_registro.nombre;
            v_total_pedidos := v_contador;
            RETURN NEXT;  -- 只有满足条件的客户才被加入结果集
        END IF;
    END LOOP;
END;
$$;
```

Cuándo: Escenarios de uso y limitaciones de rendimiento

Cuándo usar `RETURN NEXT`:

1. **Necesitas procesar fila por fila en PL/pgSQL**: Por ejemplo, necesitas llamar a otras funciones, lógica compleja o juicios condicionales dentro del bucle antes de decidir si añadir la fila al resultado.
2. **Necesitas mezclar con `RETURN QUERY`**: `RETURN NEXT` y `RETURN QUERY` pueden usarse alternativamente en la misma función, y los resultados se fusionarán en el orden de ejecución.

3. **La estructura de la función no es una consulta única**: Por ejemplo, generar datos basados en reglas de negocio complejas, o recorrer cursores explícitos.

Limitaciones de rendimiento (importante):
La implementación actual de PostgreSQL (incluida la versión 18) tiene `RETURN NEXT` y `RETURN QUERY` con una característica: **el conjunto de resultados se acumula completamente en memoria (o en disco temporal) antes de ser devuelto al llamador**. Esto significa que si tu función devuelve millones de filas, el llamador no recibirá la primera fila hasta que se hayan generado todas las filas, y la memoria podría ser insuficiente, provocando escrituras en disco, lo que afecta el rendimiento.

Recomendación: Para escenarios de grandes conjuntos de datos, prioriza usar `RETURN QUERY` (deja que el planificador de SQL haga el trabajo) en lugar de recorrer fila por fila con `FOR ... LOOP` y luego `RETURN NEXT`. `RETURN NEXT` es más adecuado para escenarios donde es necesario procesar la lógica fila por fila y el volumen de datos es controlable.

Resumen

Característica	`RETURN` (con expresión)	`RETURN NEXT`
¿Finaliza la función?	Sí, inmediatamente	**No**, continúa ejecutándose
Escenario de uso	Funciones escalares / devolver una sola fila	Funciones `SETOF` / devolver múltiples filas
Ubicación típica	Fuera del cuerpo de la función o después de un juicio condicional	Dentro de un bucle `LOOP`
Requisito de tipo de retorno	Debe coincidir con el tipo de retorno declarado	Debe coincidir con la estructura de columnas de `SETOF`

En resumen: **`RETURN NEXT` es el "añadidor de filas" en las funciones de conjunto de resultados**. Recuerda usarlo junto con un `RETURN;` sin parámetros para cerrar la función, y ten en cuenta sus limitaciones de rendimiento con grandes conjuntos de datos.

repite el ultimo prompt pero con comentarios en castellano

`RETURN NEXT` en PL/pgSQL: por qué, cómo y cuándo (con ejemplos comentados en castellano)

`RETURN NEXT` es la instrucción central de las funciones PL/pgSQL que devuelven conjuntos de resultados (`SETOF`). No "devuelve" ni "sale" de la función de inmediato; su única función es **añadir una fila al conjunto de resultados** que se está construyendo. Después de ejecutar `RETURN NEXT`, la función continúa ejecutando la siguiente instrucción hasta encontrar un `RETURN;` sin argumentos o llegar al final de la función, momento en el que se devuelve todo el conjunto de resultados acumulado.

A continuación se explica en detalle desde tres perspectivas: **por qué**, **cómo** y **cuándo**, junto con ejemplos.

Por qué: la diferencia esencial entre `RETURN NEXT` y `RETURN`

Para entender `RETURN NEXT`, primero hay que distinguirlo de `RETURN`:

- **`RETURN expresión;`**: **finaliza** la función inmediatamente y devuelve un valor. Se utiliza para funciones escalares (como `RETURNS integer`) o cuando se devuelve una sola fila.
- **`RETURN NEXT;`**: **no finaliza** la función. Solo "empaqueta" los datos de la fila actual (ya sea una variable escalar o un tipo compuesto completo) y los añade a la lista de resultados. La función continúa ejecutándose.

  Por lo tanto, si tu función está declarada para devolver `SETOF` (un conjunto de resultados), debes usar `RETURN NEXT` (o `RETURN QUERY`) para acumular filas, y finalmente usar un `RETURN;` sin parámetros para indicar el final.

Cómo: tres formas de uso comunes

Caso 1: `RETURN NEXT v_registro` (devolver una fila completa / `RECORD`)

Esta es la forma más común y eficiente. Cuando usas `FOR ... IN ... LOOP` para recorrer una consulta, la variable del bucle es esencialmente un `RECORD` (o un `%ROWTYPE`), y `RETURN NEXT` añade directamente toda la fila al conjunto de resultados.

Requisito: la estructura de la variable debe coincidir con la estructura de columnas definida por `RETURNS SETOF`.

sql

```
-- Función que devuelve todas las filas de pedidos en estado 'pendiente'.
-- Usa %ROWTYPE para que la fila completa coincida con la estructura de la
tabla.
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_pendientes()
RETURNS SETOF tienda.pedidos  -- Devuelve el conjunto de filas con la estructura
de la tabla pedidos
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro tienda.pedidos%ROWTYPE;  -- Variable de fila; nombre con prefijo
v_ según la convención
BEGIN
    -- Recorremos con un cursor implícito todos los pedidos pendientes
    FOR v_registro IN
        SELECT * FROM tienda.pedidos WHERE estado = 'pendiente'
    LOOP
        -- Añadimos la fila actual al conjunto de resultados sin salir del bucle
```

```
        RETURN NEXT v_registro;
    END LOOP;
    -- RETURN sin argumentos: cierra la función y devuelve todo lo acumulado
    RETURN;
END;
$$;
```

Ejemplo de llamada:

sql

```
-- Invocamos la función como si fuera una tabla más
SELECT * FROM tienda.fn_pedidos_pendientes();
```

Caso 2: `RETURN NEXT v_nombre, v_email` (devolver varios campos escalares)

Si tu función devuelve `RETURNS TABLE(v_nombre TEXT, v_email TEXT)`, puedes construir filas usando **parámetros de salida (`OUT`)** o **variables locales**, y usar `RETURN NEXT` para enviarlas.

sql

```
-- Función que devuelve solo dos columnas de cada cliente.
-- RETURNS TABLE define dos columnas que internamente son parámetros OUT.
CREATE OR REPLACE FUNCTION tienda.fn_listar_clientes_basico()
RETURNS TABLE (v_nombre TEXT, v_email TEXT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;  -- Variable de fila para el bucle
BEGIN
    -- Recorremos todos los clientes con un cursor implícito
    FOR v_registro IN SELECT nombre, email FROM tienda.clientes LOOP
        -- Asignamos los valores a las columnas de salida (parámetros OUT)
        v_nombre := v_registro.nombre;
        v_email  := v_registro.email;
        -- RETURN NEXT sin argumentos guarda los valores actuales de los OUT
como una fila
        RETURN NEXT;
    END LOOP;
    -- Fin de la función: se devuelve el conjunto acumulado
    RETURN;
END;
$$;
```

Nota: si la función usa `RETURNS TABLE`, los nombres de las columnas son en realidad parámetros `OUT`. En este caso, la práctica estándar es asignar valores y luego usar un `RETURN NEXT;` **sin parámetros**, que guarda los valores actuales de todas las variables `OUT`.

Caso 3: `RETURN NEXT` combinado con lógica condicional (control preciso del flujo)

Dado que `RETURN NEXT` no interrumpe la ejecución, puedes decidir dinámicamente si añadir una fila o modificar los datos antes de añadirla dentro del bucle.

sql

```
-- Función que devuelve solo los clientes que superan un número mínimo de
pedidos.
-- Demuestra que RETURN NEXT puede usarse dentro de un IF.
CREATE OR REPLACE FUNCTION tienda.fn_clientes_con_filtro(v_min_pedidos INT)
RETURNS TABLE (v_nombre TEXT, v_total_pedidos BIGINT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;        -- Fila del bucle sobre clientes
    v_contador BIGINT;        -- Contador de pedidos por cliente
BEGIN
    FOR v_registro IN SELECT cliente_id, nombre FROM tienda.clientes LOOP
        -- Calculamos cuántos pedidos tiene este cliente
        SELECT COUNT(*) INTO v_contador
        FROM tienda.pedidos
        WHERE cliente_id = v_registro.cliente_id;

        -- Solo si cumple el mínimo, añadimos la fila al resultado
        IF v_contador >= v_min_pedidos THEN
            v_nombre        := v_registro.nombre;
            v_total_pedidos := v_contador;
            RETURN NEXT;  -- Añadimos la fila; seguimos con el siguiente cliente
        END IF;
    END LOOP;
    RETURN;  -- Cierre de la función
END;
$$;
```

Ejemplo de llamada:

sql

```
-- Solo devuelve clientes con 1 o más pedidos
SELECT * FROM tienda.fn_clientes_con_filtro(1);
```

Cuándo: escenarios de uso y limitaciones de rendimiento

Cuándo usar `RETURN NEXT`:

1. **Necesitas procesar fila por fila en PL/pgSQL**: por ejemplo, necesitas llamar a otras funciones, lógica compleja o juicios condicionales dentro del bucle antes de decidir si añadir la fila al resultado.
2. **Necesitas mezclar con `RETURN QUERY`**: `RETURN NEXT` y `RETURN QUERY` pueden usarse alternativamente en la misma función, y los resultados se fusionarán en el orden de ejecución.
3. **La estructura de la función no es una consulta única**: por ejemplo, generar datos basados en reglas de negocio complejas, o recorrer cursores explícitos.

Limitaciones de rendimiento (importante):

La implementación actual de PostgreSQL (incluida la versión 18) tiene `RETURN NEXT` y `RETURN QUERY` con una característica: **el conjunto de resultados se acumula completamente en memoria (o en disco temporal) antes de ser devuelto al llamador**. Esto significa que si tu función devuelve

millones de filas, el llamador no recibirá la primera fila hasta que se hayan generado todas las filas, y la memoria podría ser insuficiente, provocando escrituras en disco, lo que afecta el rendimiento.

Recomendación: para escenarios de grandes conjuntos de datos, prioriza usar `RETURN QUERY` (deja que el planificador de SQL haga el trabajo) en lugar de recorrer fila por fila con `FOR ... LOOP` y luego `RETURN NEXT`. `RETURN NEXT` es más adecuado para escenarios donde es necesario procesar la lógica fila por fila y el volumen de datos es controlable.

Resumen

Característica	`RETURN` (con expresión)	`RETURN NEXT`
¿Finaliza la función?	Sí, inmediatamente	**No**, continúa ejecutándose
Escenario de uso	Funciones escalares / devolver una sola fila	Funciones `SETOF` / devolver múltiples filas
Ubicación típica	Fuera del cuerpo de la función o después de un juicio condicional	Dentro de un bucle `LOOP`
Requisito de tipo de retorno	Debe coincidir con el tipo de retorno declarado	Debe coincidir con la estructura de columnas de `SETOF`

En resumen: **`RETURN NEXT` es el "añadidor de filas" en las funciones de conjunto de resultados**. Recuerda usarlo junto con un `RETURN;` sin parámetros para cerrar la función, y ten en cuenta sus limitaciones de rendimiento con grandes conjuntos de datos.

ahora pone ejemplos con setof record con cursores explicitos y cursores implicitos

`SETOF RECORD` con cursores explícitos e implícitos en PL/pgSQL

`SETOF RECORD` es el tipo de retorno más flexible de PL/pgSQL: la función devuelve un conjunto de filas cuya estructura **no se declara en la función**, sino que la define **el llamador** al invocarla (con `AS t(...)`). Esto lo hace ideal para consultas dinámicas, reportes parametrizables o cuando la estructura cambia según el contexto.

La contrapartida es que el compilador **no puede verificar** que las columnas devueltas coincidan con lo que espera el llamador: si te equivocas, el error aparece en tiempo de ejecución, no al crear la función.

A continuación se muestran todas las combinaciones posibles:

- Cursor **implícito** (`FOR ... IN`) + `SETOF RECORD`
- Cursor **explícito** (`DECLARE CURSOR` / `OPEN` / `FETCH` / `CLOSE`) + `SETOF RECORD`
- Con **variables `OUT`** y **sin variables `OUT`**

- Con **%ROWTYPE**, **RECORD** y **RETURN NEXT**

---

1. Cursor implícito + SETOF RECORD sin variables OUT

En este caso la función recorre con `FOR ... IN` y usa `RETURN NEXT v_registro` para añadir cada fila entera. La estructura se define en la llamada.

sql

```
-- Cursor implícito: FOR ... IN recorre la consulta sin abrir/cerrar
manualmente.
-- Devuelve cada fila como RECORD completo; el llamador decide las columnas.
CREATE OR REPLACE FUNCTION tienda.fn_pedidos_record_implicito()
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;  -- Fila genérica que irá cambiando en cada iteración
BEGIN
    -- Cursor implícito: PostgreSQL gestiona OPEN/FETCH/CLOSE automáticamente
    FOR v_registro IN
        SELECT p.pedido_id,
               c.nombre        AS cliente,
               p.estado,
               p.fecha_pedido
        FROM tienda.pedidos p
        JOIN tienda.clientes c ON c.cliente_id = p.cliente_id
        ORDER BY p.pedido_id
    LOOP
        -- Añadimos la fila actual al conjunto de resultados
        RETURN NEXT v_registro;
    END LOOP;
    RETURN;  -- Cierre de la función
END;
$$;
```

Llamada (el llamador define la estructura con `AS t(...)`):

sql

```
SELECT *
FROM tienda.fn_pedidos_record_implicito()
AS t(v_pedido_id INTEGER, v_cliente TEXT, v_estado TEXT, v_fecha TIMESTAMP);
```

Resultado:

text

```
v_pedido_id | v_cliente    | v_estado   | v_fecha
------------+--------------+------------+---------------------
1           | Juan Pérez   | pendiente  | 2026-10-20 10:00:00
2           | María López  | pendiente  | 2026-10-20 10:05:00
```

---

2. Cursor implícito + SETOF RECORD con variables OUT

Aquí declaramos parámetros OUT (con prefijo v_ según la convención). El RETURN NEXT; sin argumentos guarda los valores actuales de los OUT.

sql

```
-- Cursor implícito con variables OUT.
-- Cada RETURN NEXT sin argumentos empaqueta los OUT actuales como una fila.
CREATE OR REPLACE FUNCTION tienda.fn_clientes_record_implicito_out(
    OUT v_cliente_id INTEGER,
    OUT v_nombre     TEXT,
    OUT v_email      TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_registro RECORD;  -- Fila del bucle
BEGIN
    FOR v_registro IN
        SELECT cliente_id, nombre, email
        FROM tienda.clientes
        ORDER BY cliente_id
    LOOP
        -- Asignamos a los OUT (nombres con prefijo v_)
        v_cliente_id := v_registro.cliente_id;
        v_nombre     := v_registro.nombre;
        v_email      := v_registro.email;
        -- Guardamos la fila con los valores actuales de los OUT
        RETURN NEXT;
    END LOOP;
    RETURN;
END;
$$;
```

Llamada (con SETOF RECORD **siempre** hay que declarar la estructura en la llamada):

sql

```
SELECT *
FROM tienda.fn_clientes_record_implicito_out()
AS t(v_cliente_id INTEGER, v_nombre TEXT, v_email TEXT);
```

Resultado:

text

```
v_cliente_id | v_nombre    | v_email
-------------+-------------+-------------------
1            | Juan Pérez  | juan@example.com
2            | María López | maria@example.com
```

---

3. Cursor explícito + SETOF RECORD sin variables OUT

El cursor explícito permite control fino: DECLARE, OPEN, FETCH, CLOSE. Es útil cuando necesitas FETCH direccional, WHERE CURRENT OF o lógica entre filas.

sql

```
-- Cursor explícito: declaramos el cursor con CURSOR FOR y lo gestionamos
manualmente.
-- Usamos %ROWTYPE para que la fila tenga la estructura de la tabla productos.
CREATE OR REPLACE FUNCTION tienda.fn_productos_record_explicito(v_umbral NUMERIC
DEFAULT 0)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_producto tienda.productos%ROWTYPE;  -- Fila con estructura fija de la
tabla
    -- Cursor explícito: la consulta puede referenciar parámetros de la función
    v_cursor CURSOR FOR
        SELECT *
        FROM tienda.productos
        WHERE precio > v_umbral
          AND activo = TRUE
        ORDER BY precio DESC;
BEGIN
    OPEN v_cursor;   -- Abrimos el cursor (ejecuta la consulta)
    LOOP
        -- Extraemos la siguiente fila; si no hay, FOUND será FALSE
        FETCH v_cursor INTO v_producto;
        EXIT WHEN NOT FOUND;

        -- Añadimos la fila entera al conjunto de resultados
        RETURN NEXT v_producto;
    END LOOP;
    CLOSE v_cursor;  -- Liberamos recursos del cursor
    RETURN;
END;
$$;
```

Llamada (el llamador debe coincidir exactamente con la estructura de `productos`):

sql

```
SELECT *
FROM tienda.fn_productos_record_explicito(50)
AS t(v_producto_id INTEGER, v_nombre TEXT, v_precio NUMERIC,
     v_stock INTEGER, v_activo BOOLEAN);
```

Resultado:

text

```
v_producto_id | v_nombre        | v_precio | v_stock | v_activo
--------------+-----------------+----------+---------+----------
3             | Monitor 27"     | 249.00   | 20      | t
1             | Teclado mecánico| 79.90    | 50      | t
```

---

4. Cursor explícito + `SETOF RECORD` con variables `OUT`

Combinamos lo mejor de ambos: control manual del cursor y salida mediante `OUT` (útil cuando queremos transformar o calcular columnas derivadas).

sql

```
-- Cursor explícito con variables OUT.
-- Calculamos el total por pedido dentro del bucle y lo devolvemos como OUT.
```

```
CREATE OR REPLACE FUNCTION tienda.fn_resumen_pedidos_record_explicito_out(
    OUT v_pedido_id INTEGER,
    OUT v_cliente   TEXT,
    OUT v_total     NUMERIC,
    OUT v_estado    TEXT
)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_fila RECORD;  -- Fila genérica del cursor
    v_cursor CURSOR FOR
        SELECT p.pedido_id, c.nombre AS cliente, p.estado,
               COALESCE(SUM(lp.cantidad * lp.precio_unit), 0) AS total
        FROM tienda.pedidos p
        JOIN tienda.clientes c    ON c.cliente_id = p.cliente_id
        LEFT JOIN tienda.lineas_pedido lp ON lp.pedido_id = p.pedido_id
        GROUP BY p.pedido_id, c.nombre, p.estado
        ORDER BY p.pedido_id;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_fila;
        EXIT WHEN NOT FOUND;

        -- Volcamos la fila del cursor a los parámetros OUT
        v_pedido_id := v_fila.pedido_id;
        v_cliente   := v_fila.cliente;
        v_total     := v_fila.total;
        v_estado    := v_fila.estado;

        RETURN NEXT;  -- Empaqueta los OUT actuales como una fila
    END LOOP;
    CLOSE v_cursor;
    RETURN;
END;
$$;
```

Llamada:

sql

```
SELECT *
FROM tienda.fn_resumen_pedidos_record_explicito_out()
AS t(v_pedido_id INTEGER, v_cliente TEXT, v_total NUMERIC, v_estado TEXT);
```

Resultado:

text

```
v_pedido_id | v_cliente    | v_total | v_estado
------------+--------------+---------+-----------
1           | Juan Pérez   | 158.90  | pendiente
2           | María López  | 249.00  | pendiente
```

---

5. Variante mixta: cursor explícito + `RETURN NEXT` de múltiples escalares

Cuando no quieres usar `OUT` pero sí devolver varias columnas calculadas, puedes declarar variables locales y pasarlas explícitamente a `RETURN NEXT v_a, v_b, ...`. Es perfectamente válido con `SETOF RECORD`.

sql

```
-- Cursor explícito que devuelve varias columnas calculadas.
-- RETURN NEXT con varios escalares equivale a una fila con esas columnas.
CREATE OR REPLACE FUNCTION tienda.fn_clientes_metricas_record(v_min_pedidos INT
DEFAULT 0)
RETURNS SETOF RECORD
LANGUAGE plpgsql
AS $$
DECLARE
    v_cliente_id   INTEGER;
    v_nombre       TEXT;
    v_num_pedidos  BIGINT;
    v_importe      NUMERIC;
    v_cursor CURSOR FOR
        SELECT cliente_id, nombre FROM tienda.clientes ORDER BY cliente_id;
BEGIN
    OPEN v_cursor;
    LOOP
        FETCH v_cursor INTO v_cliente_id, v_nombre;
        EXIT WHEN NOT FOUND;

        -- Calculamos métricas por cliente
        SELECT COUNT(*), COALESCE(SUM(lp.cantidad * lp.precio_unit), 0)
        INTO v_num_pedidos, v_importe
        FROM tienda.pedidos p
        LEFT JOIN tienda.lineas_pedido lp ON lp.pedido_id = p.pedido_id
        WHERE p.cliente_id = v_cliente_id;

        IF v_num_pedidos >= v_min_pedidos THEN
            -- RETURN NEXT con varios escalares: una fila con esas columnas
            RETURN NEXT v_cliente_id, v_nombre, v_num_pedidos, v_importe;
        END IF;
    END LOOP;
    CLOSE v_cursor;
    RETURN;
END;
$$;
```

Llamada:

sql

```
SELECT *
FROM tienda.fn_clientes_metricas_record(1)
AS t(v_cliente_id INTEGER, v_nombre TEXT, v_num_pedidos BIGINT, v_importe
NUMERIC);
```

Resultado:

text

```
v_cliente_id | v_nombre    | v_num_pedidos | v_importe
-------------+-------------+---------------+-----------
```

```
1           | Juan Pérez  | 1           | 158.90
2           | María López | 1           | 249.00
```

6. Comparativa rápida

Combinación	Declaración	Apertura	Extracción	Devolución
Implícito + sin OUT	`FOR v_registro IN ...`	Automática	Automática	`RETURN NEXT v_registro;`
Implícito + con OUT	`FOR v_registro IN ...`	Automática	Automática	Asignar OUT + `RETURN NEXT;`
Explícito + sin OUT	`CURSOR FOR ...`	`OPEN`	`FETCH ... INTO v_row`	`RETURN NEXT v_row;`
Explícito + con OUT	`CURSOR FOR ...`	`OPEN`	`FETCH ... INTO v_fila`	Asignar OUT + `RETURN NEXT;`
Explícito + escalares	`CURSOR FOR ...`	`OPEN`	`FETCH ... INTO v_a, v_b`	`RETURN NEXT v_a, v_b, ...;`

7. Reglas de oro con `SETOF RECORD`

1. **El llamador siempre debe declarar la estructura** con `AS t(...)`. Sin eso, PostgreSQL no sabe qué columnas esperar y lanza un error.
2. **Los nombres y tipos deben coincidir exactamente** con lo que devuelve la función. El compilador no lo verifica; si te equivocas, el error aparece en tiempo de ejecución.
3. **Prefiere `RETURNS TABLE` cuando la estructura sea fija**: da tipado fuerte, permite usarla en `JOIN`/`WHERE` sin `AS t(...)`, y detecta errores al crear la función.
4. **Usa `SETOF RECORD` solo cuando la estructura sea variable o desconocida** (SQL dinámico, reportes parametrizables, funciones genéricas).
5. **Con cursores explícitos, cierra siempre el cursor** (`CLOSE`) para liberar recursos; si quieres garantía total, envuelve el bucle en un bloque `BEGIN ... EXCEPTION ... END` con `CLOSE` en el handler.
6. **Todas las variables declaradas llevan el prefijo `v_`** según la convención del tutorial: `v_registro`, `v_producto`, `v_cursor`, `v_cliente_id`, etc.